RESCUE: Verified-Release Admission Control and Process Fencing under Adversarial Overload

Vishal Acharya Atharva Dhage Kausthubh Dhongade
Veermata Jijabai Technological Institute (VJTI), Mumbai, India

Abstract

Emergency communication gateways must complete legitimate requests before their deadlines while absorbing crisis-time surges and adversarial floods. Existing overload controllers release capacity when a lease expires, a request is shed, or a cancellation is issued, implicitly assuming that the underlying work has stopped. When tasks ignore cooperative cancellation, this assumption fails: the gateway's ledger reports capacity that the machine does not have (*phantom capacity*) and dispatches new work on top of lingering *zombie* tasks (*premature reuse*).

In this paper, we propose RESCUE, an admission-control and process-fencing gateway built on one principle: capacity returns to the pool only after the operating system confirms that its previous holder has exited. RESCUE runs every task in a Linux cgroup v2 cell, escalates through a five-stage termination ladder, keeps unconfirmed tasks charged in a quarantine, confines untrusted traffic to a sacrificial pool, and fences late writes with monotonic epoch tokens. We implement RESCUE in about 7,000 lines of Go and compare it with a timed-release baseline under zombie fractions of 0%, 10%, and 30%. RESCUE never allocated a slot whose previous holder was still live (the baseline produced up to 17 concurrent double allocations) and kept its accounting error within 0.03 slots (baseline: up to +9.45). This safety costs measured deadline success when zombies occupy capacity, a trade-off we quantify. FIFO start-time and drain bounds held on one million randomized workloads.

CCS Concepts: • Software and its engineering → Operating systems; • Computer systems organization → Dependable and fault-tolerant systems.

Keywords: Overload Control, Admission Control, Cancellation, Leases, Fencing, cgroups

Abbreviations and Key Terms

Table 1 lists the abbreviations used in this paper, and the terms below are used with a specific meaning throughout.

Zombie task. A task that has exceeded its deadline, ignored cooperative cancellation, or crashed silently, yet still consumes hardware (CPU cores, memory, threads).

Phantom capacity. A false accounting state in which the gateway assumes worker slots are idle because a timeout expired, while stalled processes still saturate the server.

Owner-confirmed release. Capacity is returned to the pool *only after* the operating system verifies that all related processes have terminated.

Timed release. The conventional baseline: capacity tokens are revoked as soon as a timer fires, regardless of whether execution has really stopped.

Premature reuse. Allocating a slot to a new request while the previous worker is still running on it, which causes double allocation and contention.

Sacrificial pool. A strictly bounded, sandboxed pool for first-time, untrusted, or unverified traffic, so zombies cannot exhaust legitimate emergency resources.

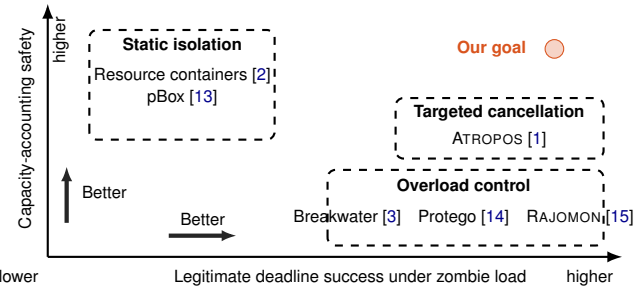


Figure 1. Conceptual design space for overload handling when tasks can resist cancellation (qualitative placement). Overload controllers assume shed work stops; isolation limits deadline success; targeted cancellation relies on cooperative termination. RESCUE aims for safe accounting while preserving legitimate deadline success.

In brief. The horizontal axis is how many legitimate requests still meet their deadline when some tasks refuse to stop (the metric S_L). The vertical axis is how truthful the gateway's capacity accounting is, i.e. whether it ever believes a slot is free while a task is still running on it. *Static isolation* gives each request a fixed share of resources. *Overload control* decides which requests to admit or shed. *Targeted cancellation* cancels the request that hogs a resource. The orange dot marks the corner RESCUE aims for: truthful accounting and high deadline success.

Termination ladder. A progressive five-stage cancellation sequence: cooperative cancel, grace period τ_1, `SIGTERM`, grace period τ_2, then `cgroup.kill`.

Fencing token. A monotonically increasing number issued with each lease; downstream stores reject writes with stale tokens to prevent out-of-order corruption.

Quarantine. A holding area for unresponsive tasks whose capacity stays charged against the server budget, preventing phantom capacity.

Orphaned-work exposure (OE). The amount of computation that keeps running after the gateway has stopped tracking it.

1 Introduction

Emergency communication gateways—flood-warning portals, disaster-reporting backends, healthcare coordination services—must work precisely when demand is least predictable. In a crisis, a surge of legitimate citizens arrives together with floods of synthetic or malformed traffic from malicious or malfunctioning clients, a form of denial of service (DoS). A report delivered after its deadline has essentially no value, so the goal is not merely to absorb packets but to ensure that scarce worker threads, memory, and database connections are spent on requests that can still finish in time.

Gateways typically bound concurrency with time-limited *leases*. When a lease expires, a client disconnects, or an

Table 1. List of abbreviations.

Abbrev.	Meaning	Abbrev.	Meaning
RESCUE	Resource-safe Emergency Scheduling and Control Under Overload	**ASN**	Autonomous System Number (routing-domain identifier of a network source)
OC	Owner-Confirmed Capacity Release: release only after process death is verified	**NAT**	Network Address Translation (many devices share one IP address)
TR	Timed Release: conventional policy that frees capacity when the lease timer expires	**RPC**	Remote Procedure Call (inter-service network invocation)
cgroup v2	Linux Control Groups version 2: kernel-level hierarchical process isolation	**RPS**	Requests Per Second (throughput measure)
FIFO	First-In, First-Out (scheduling order)	**IPC**	Inter-Process Communication
FSM	Finite State Machine (model of the lease state transitions)	**SIGTERM**	Unix signal asking a process to terminate gracefully
S_L	Legitimate Deadline Success rate: share of real requests that meet their deadline	**SIGKILL**	Unix signal that kills a process immediately; it cannot be caught
OE	Orphaned-Work Exposure: lingering, unmonitored computation	**DoS**	Denial of Service (resource-exhaustion attack)
QoS	Quality of Service		

HTTP request times out, the gateway marks the slot free and dispatches the next waiting request. We call this policy *timed release* (TR). TR is safe only if an expired task actually stops. In practice, tasks routinely ignore cooperative cancellation because of uninterruptible system calls, blocking third-party libraries, or adversarial code paths. We call such a task a *zombie*. When a zombie outlives its lease, the gateway's ledger and the physical machine disagree: the ledger believes a slot is idle while the zombie still consumes CPU, memory, and threads. We call this gap *phantom capacity*. Dispatching a fresh request onto the slot, which we call *premature reuse*, creates contention that grows with every additional zombie and can cascade into a gateway-wide collapse.

Existing overload-handling strategies fall broadly into three categories (Figure 1). *Admission control and load shedding*, such as Breakwater [3], Protego [14], and RAJOMON [15], decide which requests to admit or drop, but they treat a dropped request as one that no longer consumes resources. *Targeted cancellation*, represented by ATROPOS [1], identifies the request responsible for contention and cancels it through the application's own cancellation initiator. This is effective when tasks cooperate: ATROPOS reports that 76% of 151 surveyed applications already implement cancellation, and that its policy sustains 96% of baseline throughput on 16 reproduced overload cases [1]. But cooperative cancellation is a request, not a guarantee; ATROPOS itself notes that forced thread-level cancellation is unsafe and keeps it disabled by default. *Performance isolation*, such as resource containers [2] and pBox [13], partitions resources among requests, but static partitions limit utilization and do not terminate work that has already outlived its budget.

These limitations raise two questions. **(Q1)** *When tasks outlive their deadlines or ignore cancellation under adversarial overload, can a gateway enforce no premature capacity reuse without introducing global deadlock?* **(Q2)** *Can completion-backed accounting prevent phantom capacity while preserving legitimate deadline success (S_L) relative to timed-release baselines?*

To answer them, we design RESCUE, whose core principle is *owner-confirmed release* (OC): a slot is returned to the pool only when the operating system independently confirms that every process of the previous tenant has exited. RESCUE does not trust the task, the timer, or its own bookkeeping to decide that capacity is free. Three ideas make this practical:

- **Verified release.** Each task runs in its own cgroup v2 cell. An escalating five-stage termination ladder ends in the kernel's `cgroup.kill`, and the slot is released only when `cgroup.events` reports `populated 0`.
- **Conservative accounting.** A task that does not exit moves to a quarantine where its capacity stays charged against its origin pool, so the ledger can only err on the side of less capacity, never more.
- **Physical containment.** Untrusted traffic runs in a bounded sacrificial pool, so zombies created by an attacker cannot consume the normal or emergency pools; monotonic fencing tokens stop zombies from corrupting downstream state with late writes.

We implement RESCUE as a Go gateway of about 7,000 lines across 13 packages, with a Python harness for verification. Across nominal, moderate (10% zombies), and severe (30% zombies) workloads, RESCUE records zero premature reuse, while the TR baseline records 6 and 17 concurrent double allocations in the moderate and severe cases and an accounting error of $+3.00$ and $+9.45$ slots. RESCUE's accounting error stays within -0.03. The price is deadline success: S_L drops to 0.7677 and 0.2286 under moderate and severe overload, because capacity held by unconfirmed zombies is no longer lent out.

In summary, this paper makes the following contributions:

- We identify premature reuse and phantom capacity as failure modes of timed-release and cooperative-cancellation controllers, and state five runtime invariants that rule them out.
- We propose owner-confirmed release and design RESCUE, an eight-component gateway combining tiered

pools, cgroup v2 execution cells, quarantine charging, fencing, and an independent reverse audit.
- We derive deterministic FIFO start-time and Graham-style drain bounds for bounded dispatch, and check them on one million randomized instances.
- We implement RESCUE and evaluate it against a timed-release baseline, reporting both its safety gains and its deadline-success cost.

Scope. This work covers (i) the specification and implementation of an eight-component admission and execution gateway in Go; (ii) Linux kernel-level process isolation using cgroup v2 execution cells with an authoritative five-stage cancellation ladder; (iii) an empirical comparison against a timed-release baseline under nominal, moderate, and severe zombie workloads, with a direct comparison to RAJOMON and ATROPOS left for future work; and (iv) mathematical bounds and model checking for FIFO start time and drain time.

Organization. Section 2 gives background and motivation and reviews existing systems. Section 3 presents the RESCUE design, invariants, and formal bounds. Section 4 describes the Go and cgroup implementation. Section 5 reports the experimental results. Sections 6–8 cover related work, limitations, and conclusions.

2 Background and Motivation

2.1 Zombie Tasks and Phantom Capacity

A gateway with K execution slots keeps a *ledger* of which slots are free. We say a task is a *zombie* if it has passed its deadline or lease, ignored a cooperative cancellation signal, or hung silently, yet still consumes CPU, memory, or threads. *Phantom capacity* is the difference between the capacity the ledger reports free and the capacity that is physically free. We measure it as the *accounting error*, positive when the ledger over-reports.

Illustrative scenario. Consider $K=8$ slots with a 100 ms lease. Two admitted tasks block in an uninterruptible call and do not observe cancellation. At $t=100$ ms, a TR gateway frees both slots and admits two new requests. The machine now runs ten tasks on eight slots while the ledger shows eight busy and no discrepancy. Each additional zombie widens the gap, so a modest fraction of cancellation-resistant requests (10% in our moderate workload) is enough to cause sustained oversubscription. The new requests, which are legitimate, are slowed by contention with tasks the gateway believes are gone.

Adversarial amplification. Phantom capacity is not only a bug but also an attack surface. An adversary needs only to craft requests that resist cancellation. Because TR recycles slots on a timer, each such request buys the attacker more concurrency than the gateway intended to grant, and the extra load lands on slots shared with legitimate emergency traffic.

2.2 Limitations of Existing Solutions

Table 2 contrasts the designs most related to our setting, based on our reading of the published systems.

Load shedding. RAJOMON [15] attaches tokens to requests, charges a price per API, and drops requests with insufficient tokens, propagating prices through the call graph. It controls how much work enters the system, but a request that was admitted and then outlives its budget is outside its scope. Breakwater [3] and Protego [14] similarly manage admission using queuing delay or lock-wait signals and do not verify that dropped or timed-out work has physically stopped.

Targeted cancellation. ATROPOS [1] improves on indiscriminate dropping by canceling the task with the largest estimated resource gain, using the application's existing cancellation initiator. Its design assumes that the initiator succeeds: it triggers the application's built-in mechanism and, to our reading of the paper, does not independently verify at the OS level that the task has stopped. It also limits each task to one cancellation and re-executes canceled tasks later to avoid starvation. These are sound choices for cooperative software. For adversarial or buggy tasks that never reach a cancellation checkpoint, a verified release step is a missing complement; indeed, RESCUE's ladder can be seen as a stronger cancellation backend for tasks that do not cooperate.

Leases. Leases [8] bound how long a holder may use a resource, and fencing tokens [4, 10] prevent a stale holder from corrupting state after its lease has lapsed. Leases give time bounds on *authorization*, not on physical *execution*: a lapsed holder can keep running. RESCUE couples the lease to a physical confirmation.

2.3 Challenges

Closing the gap between ledger and machine raises four challenges.

C1: Safety without deadlock. Refusing to reuse a slot until its holder exits means a truly stuck task holds capacity indefinitely. The gateway must stay live for the rest of the system.

C2: An authoritative exit signal. Process-table scans, signals, and timers can each miss child processes or race with forks. Confirmation must cover every process the task spawned.

C3: Containing untrusted load. Under a flood from unknown sources, zombies will exist. They must be confined so they cannot starve legitimate traffic.

C4: Stale side effects. Even after a lease lapses, a zombie may complete and issue a late write. Downstream state must reject it.

Table 2. Comparison of overload-handling designs (based on the cited papers).

	RAJOMON [15]	ATROPOS [1]	RESCUE (this paper)
Overload mechanism	Per-API token pricing and load shedding	Cancel the task with the highest estimated resource gain	Tiered pools, quotas, leases, bounded FIFO dispatch
How a task is stopped	Not addressed (drop at ingress)	Application's cancellation initiator (cooperative)	Five-stage ladder ending in `cgroup.kill`
Capacity considered free	When the request is dropped or completes	When the application reports the tracked resource released	Only when `cgroup.events` reports `populated 0`
Unresponsive tasks	No containment	May keep executing; one cancellation per task	Quarantine, capacity stays charged
Downstream state	Not addressed	Relies on application-level cancellation safety	Monotonic fencing tokens at the sink

2.4 Why OS-Level Confirmation Is Feasible

Linux cgroup v2 [9] provides the primitives RESCUE needs. Each cgroup exposes a `cgroup.events` file whose `populated` key becomes 0 only when no process remains in the cgroup or its descendants, and, since Linux 5.14, a write-only `cgroup.kill` file that sends `SIGKILL` to all of them. A task spawned inside a cell cannot escape it by forking. One caveat shapes our design: `SIGKILL` cannot terminate a process blocked in uninterruptible sleep until its system call returns. A kill is therefore not a confirmation, and some tasks will remain populated after the ladder completes. This is exactly why RESCUE needs a quarantine rather than assuming the kill succeeded.

3 Design of RESCUE

RESCUE is an admission-control and process-fencing gateway. Its design is organized around five runtime invariants (Section 3.1) and eight components. Figure 2 shows the path of a request.

The eight components. (1) intake and source classification, (2) admission control and quotas, (3) multi-tier execution pools, (4) reservations, leases, and bounded dispatch, (5) execution cells and release confirmation, (6) the quarantine engine, (7) fencing and effect protection, and (8) reverse audit and measurement. Figure 3 contrasts the release rule with timed release.

3.1 Invariants

RESCUE enforces five invariants at runtime.

- **I1 (No premature reuse).** No slot or capacity budget is allocated while a prior holder's process handle is unconfirmed or running.
- **I2 (Ledger–auditor agreement).** The ledger never marks a task released if the independent OS auditor observes live child processes.
- **I3 (Quarantine charging).** Quarantined tasks remain charged to their origin pool: $K_{\text{avail}} = K - |\text{quarantined}|$.
- **I4 (Bounded pools and queues).** No queue or pool depth exceeds its configured bound.
- **I5 (Fencing monotonicity).** A write carrying an obsolete or regressed epoch token is permanently rejected by the sink.

I1–I3 address phantom capacity (C1, C2), I4 bounds the cost of any single flood (C3), and I5 addresses stale effects (C4).

3.2 Intake, Classification, and Quotas

Each incoming request is stamped with a cryptographic receipt and classified into a trust tier: *Probationary*, *Established*, or *Emergency*. Concurrency is capped hierarchically at the source, autonomous-system (ASN), and global scopes (8, 16, and 64 in our prototype configuration), which limits what any single source or network can hold even when many clients share an address through NAT. Newly seen or unauthenticated sources start Probationary and cannot promote themselves; promotion requires explicit administrative attestation. Admission applies a deterministic reservation policy (Policy B) over bounded FIFO queues and rejects any request that exceeds its quota immediately with `RESOURCE_EXHAUSTED`, so overload is signaled at the door rather than inside the queue.

3.3 Tiered Execution Pools

Total concurrency is partitioned into three isolated pools:

$$K = K_{\text{sacrificial}} + K_{\text{normal}} + K_{\text{emergency}}. \tag{1}$$

Probationary traffic runs only in the sacrificial pool. If an attacker fills it with zombies, the normal pool and the protected emergency floor are unaffected. Each task is charged to its *origin pool*, and capacity always returns there.

Property 1 (Containment)**.** *Under I1–I4, zombies created by Probationary traffic can reduce only* $K_{sacrificial}$.

This is a direct consequence of routing and charging: Probationary tasks are never dispatched outside the sacrificial pool, and a quarantined task is charged to the pool it came from. It also answers Q1's deadlock concern. A pool whose slots are all held by unkillable tasks does stall, but the stall is confined to that pool and bounded by I4, and the rest of the gateway continues to make progress.

3.4 Leases and Fencing Tokens

Dispatch issues a time-bounded lease together with an epoch fencing token, a monotonically increasing integer. A lease moves through a seven-state lifecycle whose transition rules we verify with a finite-state-machine checker (Section 4). The lease is an *authorization* with a deadline; it is the cell, not the lease, that decides when capacity is free.

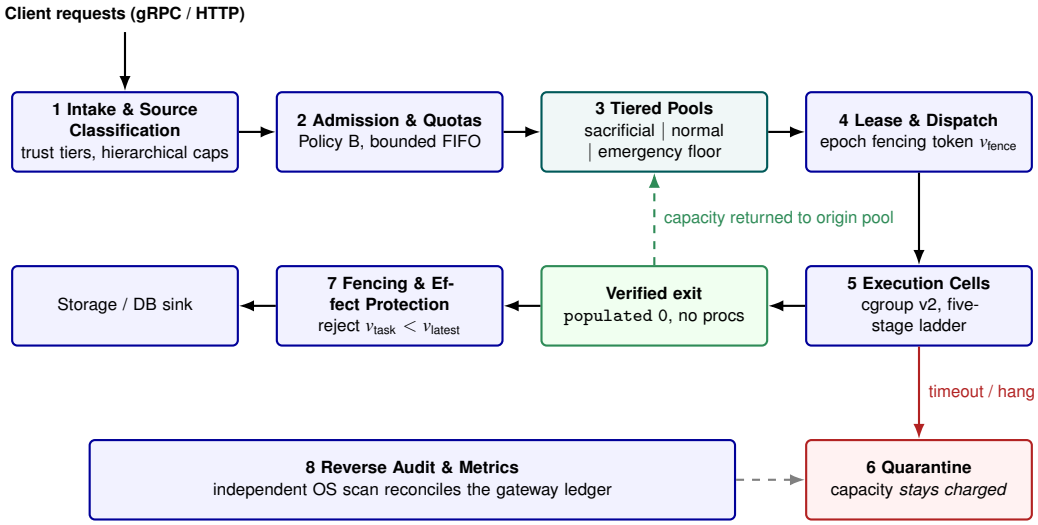


Figure 2. Overview of RESCUE. Requests are classified (1), admitted against quotas (2), assigned to a tiered pool (3), leased with a fencing token (4), and run in a cgroup v2 cell (5). Capacity is returned to the origin pool only on verified exit; tasks that do not exit are quarantined (6) with their charge retained. Writes carry tokens that the sink checks (7), and an independent auditor reconciles ledger and kernel state (8).

In brief. Follow the arrows left to right along the top row, then down. **(1) Intake** labels each request with a trust tier (Probationary, Established, Emergency) and limits how many requests one source, one network (ASN), and the whole system may have in flight. **(2) Admission** applies the quota policy and rejects excess requests immediately. **(3) Tiered pools** split the slots into sacrificial (untrusted), normal, and a protected emergency floor. **(4) Lease & dispatch** gives an admitted request a time-limited permit plus a fencing token v_{fence}. **(5) Execution cells** run the task as an isolated OS process group inside a cgroup v2. If it exits and the kernel confirms this ("Verified exit"), its slot goes back to the pool it came from. If it hangs, it moves to **(6) Quarantine**, where its slot remains counted as used. **(7) Fencing** makes the storage sink refuse writes whose token is older than the latest one ($v_{\text{task}} < v_{\text{latest}}$), and **(8) Reverse audit** independently scans the operating system's process list and compares it with the gateway's own records.

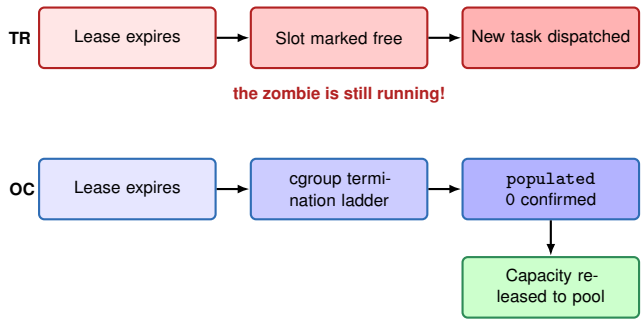


Figure 3. Timed release (TR) versus RESCUE's owner-confirmed release (OC).

In brief. In TR (top), the *timer* decides that a slot is free, so the next request is dispatched even if the old task is still executing. In OC (bottom), the timer only starts the cancellation ladder; the slot is freed after the kernel reports that the task's cgroup has no processes left (`populated 0`).

```
func ReleaseSlot(cell, lease):
  cell.cooperativeCancel()          // (1)
  if waitExit(cell, tau1): goto confirm
  killpg(cell, SIGTERM)             // (3)
  if waitExit(cell, tau2): goto confirm
  write("1", cell + "/cgroup.kill") // (5)
  if !waitExit(cell, confirmDeadline):
      quarantine.add(lease)         // stays charged
      return
confirm:
  if populated(cell) == 0 && procs(cell).empty():
      pool[lease.origin].release()  // the only release
  else:
      quarantine.add(lease)
```

Figure 4. Owner-confirmed release (simplified). Capacity returns to the origin pool on exactly one path.

In brief. The routine escalates from polite to forceful. It asks the task to stop, waits (τ_1), sends `SIGTERM`, waits (τ_2), then makes the kernel kill the whole cell. At every step, if the task has exited, it jumps to `confirm`. At `confirm`, the slot is freed only if the kernel reports no processes remain (`populated` is 0 and the process list is empty). Otherwise the task goes to quarantine and its slot stays charged.

3.5 Execution Cells and the Termination Ladder

Rather than run work as goroutines, which the Go runtime cannot forcibly stop, RESCUE launches each task as a separate OS process inside its own cgroup v2 directory (for example, `/sys/fs/cgroup/rescue/task_102`). When a task's lease expires or its request is cancelled, RESCUE executes a five-stage ladder:

(1) **Cooperative cancel** through the task's IPC channel.

(2) **Grace period** τ_1 (50 ms in our prototype) for normal cleanup.

(3) `SIGTERM` to all process groups in the cell.

(4) **Grace period** τ_2 (50 ms) for signal handlers.

(5) `cgroup.kill`, which kills every process in the cell.

Each rung is a bounded wait for exit, so well-behaved tasks leave early and only stubborn ones reach the kernel kill. Figure 4 gives the release routine. The slot is released if and only if `cgroup.events` reads `populated 0` and `cgroup.procs` is empty. Anything else, including a task stuck in uninterruptible sleep after the kill, goes to quarantine.

3.6 Quarantine

The quarantine holds every task whose exit the kernel has not confirmed. Its entries keep their capacity charge on the origin pool (I3), so K_{avail} decreases by one per quarantined task. The quarantine is cleaned when the blocked call finally returns and `populated` reaches 0; only then is the charge

removed. The ledger can therefore under-report capacity while a task is stuck, but it can never over-report.

3.7 Fencing and Effect Protection

A zombie may finish after its lease lapsed and attempt to commit a result. Each write carries the task's epoch token v_{task}, and the storage sink rejects any write with $v_{\text{task}} < v_{\text{latest}}$ (I5). This follows the fencing-token pattern used with lock services [4, 10], applied here to gateway leases so that a stale executor cannot overwrite the work of its successor.

3.8 Reverse Audit

Ledger-based safety is only as good as the ledger. An independent auditor therefore scans the real OS process table and compares it with the gateway's records. Any process in the kernel's view that the ledger believes is released is flagged (I2), which catches untracked orphans, accounting bugs, and escapes from the cell hierarchy. The auditor also emits the metrics used in Section 5.

3.9 Bounded Dispatch and Scheduling Bounds

Admission and dispatch are only useful if admitted requests have predictable waits. With each task's wall-clock cap enforced by the ladder, RESCUE obtains two deterministic bounds for a stage of m identical workers serving a FIFO queue without idling while work is queued.

Theorem 1 (FIFO start-time bound)**.** *Let request j arrive at t_a and start at S_j. Let held_live be the total processing cap of all admitted tasks live in the stage at t_a, including j, and β_j the cap of j. Then*

$$S_j - t_a \le \frac{held_live - \beta_j}{m}. \tag{2}$$

Proof sketch. Until j starts, all m workers are busy. The work they complete in $[t_a, S_j]$ is at most the cap of the tasks ahead of j, namely $held_live - \beta_j$, so $m(S_j - t_a) \le held_live - \beta_j$. □

Theorem 2 (Drain bound)**.** *Let held_live be the total cap of all pending and executing tasks and $\beta_{\max}$ the largest individual cap. The time $C_{\max}$ to drain them satisfies*

$$C_{\max} \le \frac{held_live}{m} + \left(1 - \frac{1}{m}\right)\beta_{\max}. \tag{3}$$

Proof sketch. This is Graham's list-scheduling argument [7]. Let ℓ be the last task to finish. No worker idles before S_ℓ, so $S_\ell \le (held_live - \beta_\ell)/m$ and $C_{\max} = S_\ell + \beta_\ell \le held_live/m + (1 - 1/m)\beta_\ell \le held_live/m + (1 - 1/m)\beta_{\max}$. □

Quarantined tasks reduce the number of usable workers, so the bounds apply with m replaced by $m - |\text{quarantined}|$ for the affected pool.

4 Implementation

Gateway. We implemented RESCUE in Go 1.22+ with about 7,000 lines of code organized into 13 packages: `intake`, `source`, `admission`, `queue`, `pool`, `lease`, `task`, `worker`, `quarantine`, `audit`, `effect`, `metrics`, and `controller`. The `worker` package owns the cgroup v2 lifecycle: it creates the cell directory, spawns the task as an OS process inside it, drives the ladder, and reads `cgroup.events` and `cgroup.procs` for confirmation. The `effect` package implements the fencing layer that validates writes against the current epoch. The benchmark driver is `gateway/cmd/rescue-eval`.

Platform requirements. Verified release needs Linux with cgroup v2 and, for `cgroup.kill`, kernel 5.14 or newer. On hosts without cgroups (for example, Windows or macOS), the prototype falls back to in-memory process emulation, which preserves the protocol logic but not the physical guarantee.

Verification harness. A Python 3.12 harness checks the design offline: `scheduling_bound_test.py` evaluates the two scheduling bounds on one million randomized instances; `checker_differential_test.py` runs a differential checker over 100,000 transactional histories with planted stale writes; and `lease_ownership_fsm.py` checks the protocol rules over the seven-state lease lifecycle.

5 Evaluation

We evaluate RESCUE to answer: (1) Does it eliminate premature reuse (H1)? (2) Does it keep the ledger consistent with physical load (H2)? (3) What does safety cost in legitimate deadline success and latency (H3)? (4) Do the scheduling bounds hold?

5.1 Experimental Setup

Workloads. Matched workloads are run through `rescue-eval` in three scenarios: *Nominal* (40 rps offered, 0% zombies), *Moderate* (60 rps, 10% zombies), and *Severe* (80 rps, 30% zombies). A zombie request ignores cancellation and keeps executing past its lease.

Baseline. The baseline runs the same gateway with owner-confirmed release replaced by timed release: capacity is returned as soon as the lease expires. This captures the release semantics shared by timed-release and cooperative-cancellation controllers such as those in Table 2. It is not a re-implementation of RAJOMON or ATROPOS, and we do not claim a head-to-head result against them here.

Metrics. S_L is the fraction of legitimate requests completed by their deadline among those offered. *Max unsafe error* is the peak number of slots simultaneously allocated while a previous holder was still live, as observed by the independent auditor. *Accounting error* is the average difference between ledger-free and physically free capacity (positive means phantom capacity). We also report p95 latency of completed requests.

[TODO: add hardware (CPU/RAM/kernel), run duration, number of repetitions, and variance or confidence intervals for Table 3]

5.2 Safety: No Premature Reuse (H1)

Table 3 and Figure 5(b) show that RESCUE's max unsafe error is 0 in all three scenarios, consistent with I1. The timed-release baseline allocates capacity onto live holders:

Table 3. RESCUE (OC) versus the timed-release baseline (TR) under zombie load.

Scenario	System	Offered	Zombies	S_L	Max unsafe	Acct. error	p95 (ms)
Nominal	RESCUE	40 rps	0%	1.0000	0	−0.02	73.4
	Baseline	40 rps	0%	1.0000	0	−0.03	74.9
Moderate	RESCUE	60 rps	10%	0.7677	0	−0.02	113.4
	Baseline	60 rps	10%	1.0000	6	+3.00	73.5
Severe	RESCUE	80 rps	30%	0.2286	0	−0.03	118.2
	Baseline	80 rps	30%	0.9952	17	+9.45	98.9

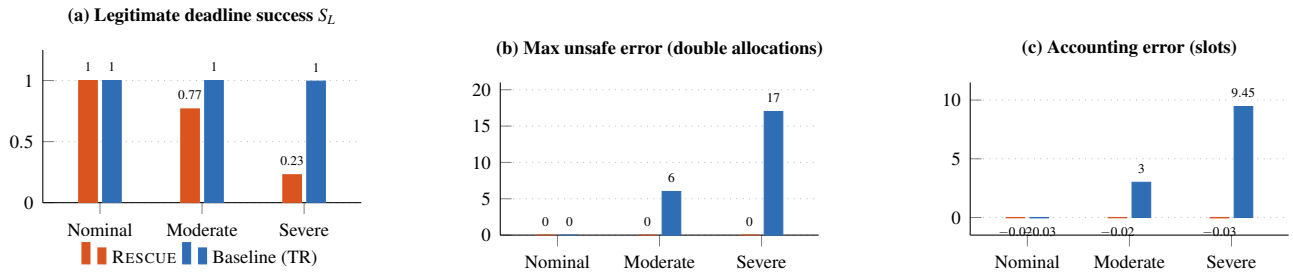


Figure 5. Safety and success under zombie load. RESCUE eliminates double allocation and phantom capacity (b, c) but delivers lower S_L than the baseline when zombies occupy capacity (a).

In brief. Each group of bars is one workload (more zombie requests from left to right). Orange bars are RESCUE, blue bars the timed-release baseline. Panel (a) is the share of legitimate requests that met their deadline; higher is better. Panel (b) counts slots given to a new request while the old one was still running; zero is safe. Panel (c) is how far the gateway's idea of free capacity is from reality; positive means it thinks slots are free that are not, and values near zero are best.

6 concurrent double allocations under moderate load and 17 under severe load. Without zombies (Nominal), both systems are safe, which indicates the baseline's failures are caused by cancellation-resistant tasks and not by any other defect of the harness.

5.3 Zombie Containment and Phantom Capacity (H2)

Under 30% zombies, the baseline's accounting error reaches +9.45: on average the controller believed nearly ten more slots were free than physically were. RESCUE's error stays at −0.03, slightly negative because quarantined or still-draining tasks are charged until the kernel confirms their exit. The small negative value is the intended direction of error under I3: the ledger may under-report capacity but should never over-report it.

5.4 Legitimate Deadline Success and Latency (H3)

Safety is not free. At moderate load RESCUE's S_L is 0.7677 against the baseline's 1.0000, and at severe load 0.2286 against 0.9952; p95 latency rises from 73.5 to 113.4 ms and from 98.9 to 118.2 ms. At nominal load, the two systems are comparable (73.4 versus 74.9 ms; S_L of 1.0000 for both).

The baseline's higher S_L should not be read as better service. It is obtained by dispatching onto occupied capacity, which the auditor flags as unsafe in 6 and 17 cases. In RESCUE, capacity held by zombies stays unavailable until the OS confirms exit, so fewer slots serve legitimate requests exactly when zombies are most numerous. We therefore view the results as an honest operating point on a safety–throughput trade-off. Pool sizing, the grace periods τ_1 and τ_2, and the size of the emergency floor are the levers that move this point, and we have not yet tuned them.

5.5 Validation of the Scheduling Bounds and Fencing

We evaluated Theorems 1 and 2 on one million randomized workload instances with Uniform, Pareto, and Bimodal processing-time profiles and observed zero violations. Randomized testing complements the proof sketches and guards against implementation slips. The differential checker detected all planted stale writes in 100,000 transactional histories, supporting I5 at the checker level. This is not yet an end-to-end test against a live database.

6 Related Work

Overload control. Breakwater [3] adjusts admission from observed queuing delay for microsecond-scale RPCs; Protego [14] targets lock contention by dropping requests whose lock waits approach the SLO; adaptive overload control for Internet servers [16] and priority-based admission in WeChat's DAGOR [5] bound or prioritize incoming work. RAJOMON [15] extends this to microservice call graphs using prices and tokens. These systems decide what to admit; RESCUE additionally decides when admitted work has *verifiably* left. Receive livelock [11] is an earlier example of a gateway that does work that does not translate into progress.

Cancellation. ATROPOS [1] selects culprit requests via resource contention and gain estimates, and cancels them through each application's existing mechanism. RESCUE is complementary: its ladder and confirmation step could serve as a cancellation backend when cooperative mechanisms are absent or untrusted, and ATROPOS's culprit selection could inform which tasks RESCUE prioritizes for early termination.

Performance isolation. Resource containers [2], PARTIES [12], DARC [6], and pBox [13] partition or schedule resources to limit interference. RESCUE's pools are also a form of partitioning, but they are used to contain unverified work, and capacity accounting is tied to kernel-confirmed exit.

Leases and fencing. Leases [8] and lock services such as Chubby [4] bound the validity of a grant, and fencing tokens [10] protect storage from stale holders. RESCUE applies both to a gateway and adds physical confirmation of exit.

7 Limitations and Future Work

Scope of the baseline. Our evaluation compares against a timed-release variant of the same gateway. A direct comparison with RAJOMON and ATROPOS on identical servers, workloads, and resource budgets remains to be done.

Cost of safety. The S_L drop in Table 3 is substantial under heavy zombie load. Tuning the ladder, pool partitions, and emergency floor, and reclaiming quarantined capacity faster, are the next steps. The fork and context-switch overhead of per-task processes was not isolated in these experiments.

Platform and stuck tasks. Verified release requires Linux with cgroup v2. Tasks in uninterruptible sleep cannot be killed until the call returns, so a pool can stall (Property 1) even though it does so safely.

Fencing. Fencing is validated by a differential checker. Extending verified release to stateful backend handles, such as dedicated PostgreSQL connections, is a primary direction. Others include eBPF probes for sub-millisecond process accounting and multi-region failover.

8 Conclusion

Overload controllers that release capacity on a timer or a cancellation request carry an unstated assumption: that the work has stopped. Under cancellation-resistant, adversarial load that assumption is false, and it produces phantom capacity and premature reuse. RESCUE replaces it with owner-confirmed release: a kernel-confirmed exit gates every reuse of capacity, unconfirmed tasks stay charged in quarantine, untrusted traffic is confined to a sacrificial pool, and fencing tokens protect downstream state. In our experiments this eliminated double allocation and kept the ledger faithful to the machine, at a measurable cost in deadline success under heavy zombie load. We believe conservative accounting is the right default for emergency systems, and quantifying and reducing its cost is the work ahead.

References

[1] Y. Hu, Z. Zhang, Y. Liu, Y. Gu, S. Lei, B. Kasikci, and P. Huang. Mitigating application resource overload with targeted task cancellation. In *Proc. ACM SOSP*, 2025. doi:10.1145/3731569.3764835.

[2] G. Banga, P. Druschel, and J. C. Mogul. Resource containers: A new facility for resource management in server systems. In *Proc. USENIX OSDI*, 1999.

[3] I. Cho, A. Saeed, J. Fried, S. J. Park, M. Alizadeh, and A. Belay. Overload control for μs-scale RPCs with Breakwater. In *Proc. USENIX OSDI*, 2020.

[4] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proc. USENIX OSDI*, 2006.

[5] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu, R. Gu, B. C. Ooi, and J. Yang. Overload control for scaling WeChat microservices. In *Proc. ACM SoCC*, 2018.

[6] H. M. Demoulin, J. Fried, I. Pedisich, M. Kogias, B. T. Loo, L. T. X. Phan, and I. Zhang. When idling is ideal: Optimizing tail-latency for heavy-tailed datacenter workloads with Perséphone. In *Proc. ACM SOSP*, 2021.

[7] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.*, 17(2):416–429, 1969.

[8] C. Gray and D. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proc. ACM SOSP*, 1989.

[9] T. Heo. Control group v2. Linux kernel documentation, kernel.org.

[10] M. Kleppmann. How to do distributed locking. Blog post, 2016.

[11] J. C. Mogul and K. K. Ramakrishnan. Eliminating receive livelock in an interrupt-driven kernel. *ACM Trans. Comput. Syst.*, 15(3):217–252, 1997.

[12] S. Chen, C. Delimitrou, and J. F. Martinez. PARTIES: QoS-aware resource partitioning for multiple interactive services. In *Proc. ACM ASPLOS*, 2019.

[13] Y. Hu, G. Huang, and P. Huang. Pushing performance isolation boundaries into application with pBox. In *Proc. ACM SOSP*, 2023.

[14] I. Cho, A. Saeed, S. J. Park, M. Alizadeh, and A. Belay. Protego: Overload control for applications with unpredictable lock contention. In *Proc. USENIX NSDI*, 2023.

[15] J. Xing, A. Giannoukos, P. Loh, S. Wang, J. Qiu, H. M. Demoulin, K. Kallas, and B. C. Lee. Rajomon: Decentralized and coordinated overload control for latency-sensitive microservices. In *Proc. USENIX NSDI*, 2025.

[16] M. Welsh and D. Culler. Adaptive overload control for busy Internet servers. In *Proc. USENIX USITS*, 2003.